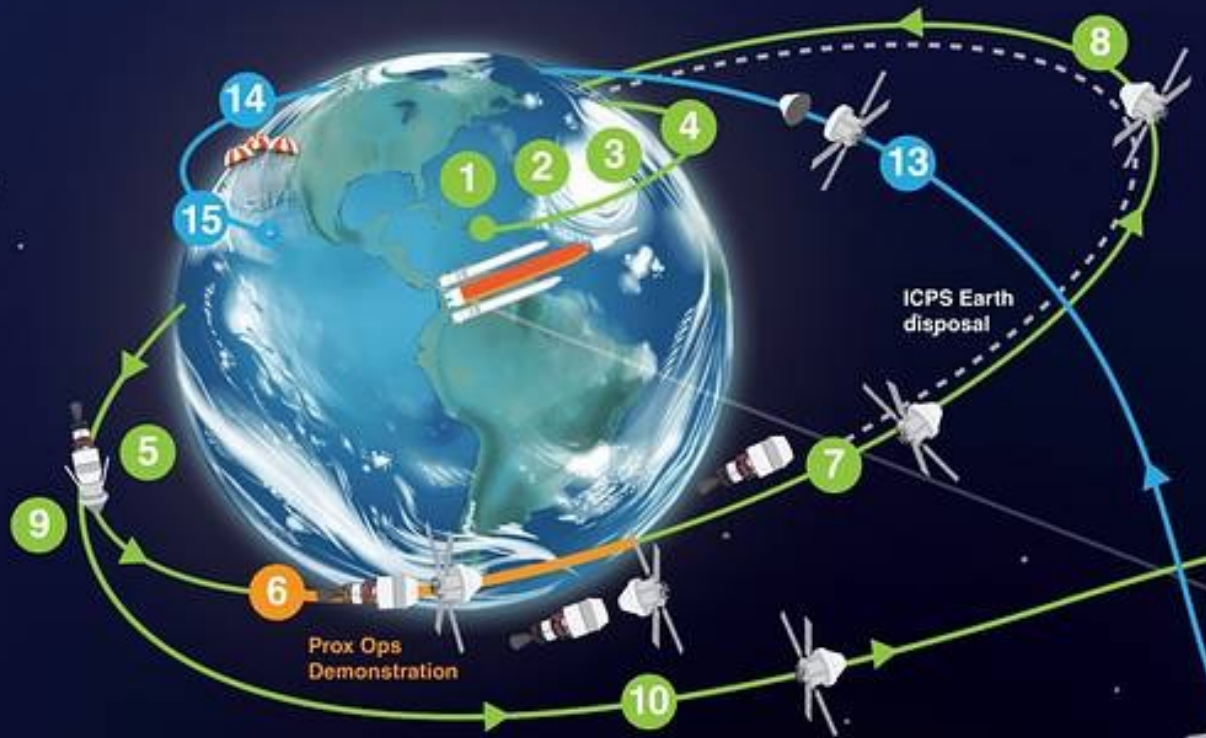
MATLAB Final Project – Stage 1

APMA 2501 - Spring 2026

Heze Chen
hc9xc@virginia.edu

Topic for today:

- Gravity and Orbital Mechanics



ARTEMIS II

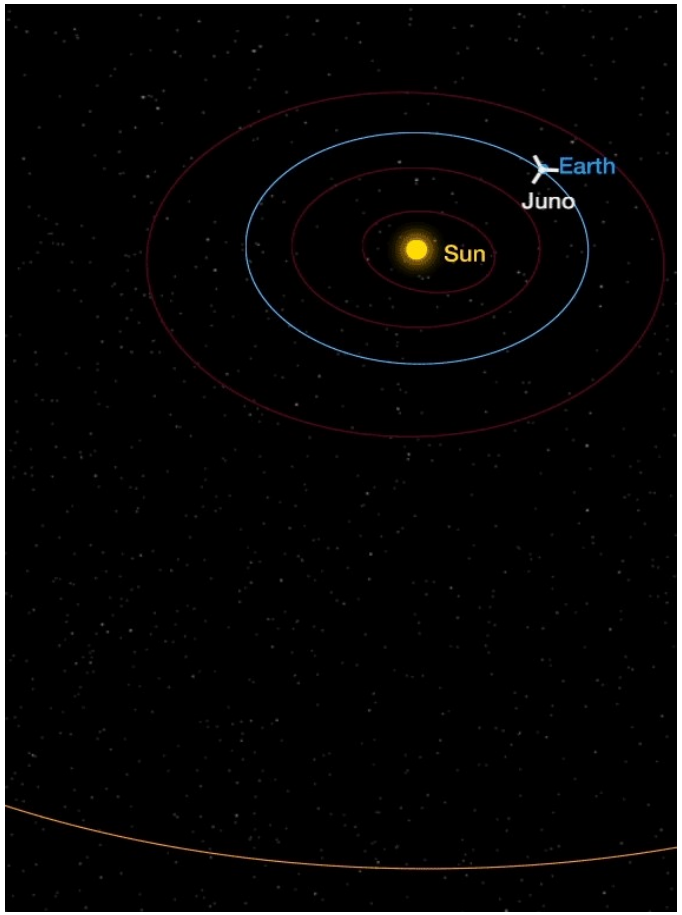
First Crewed Test Flight to the Moon Since Apollo

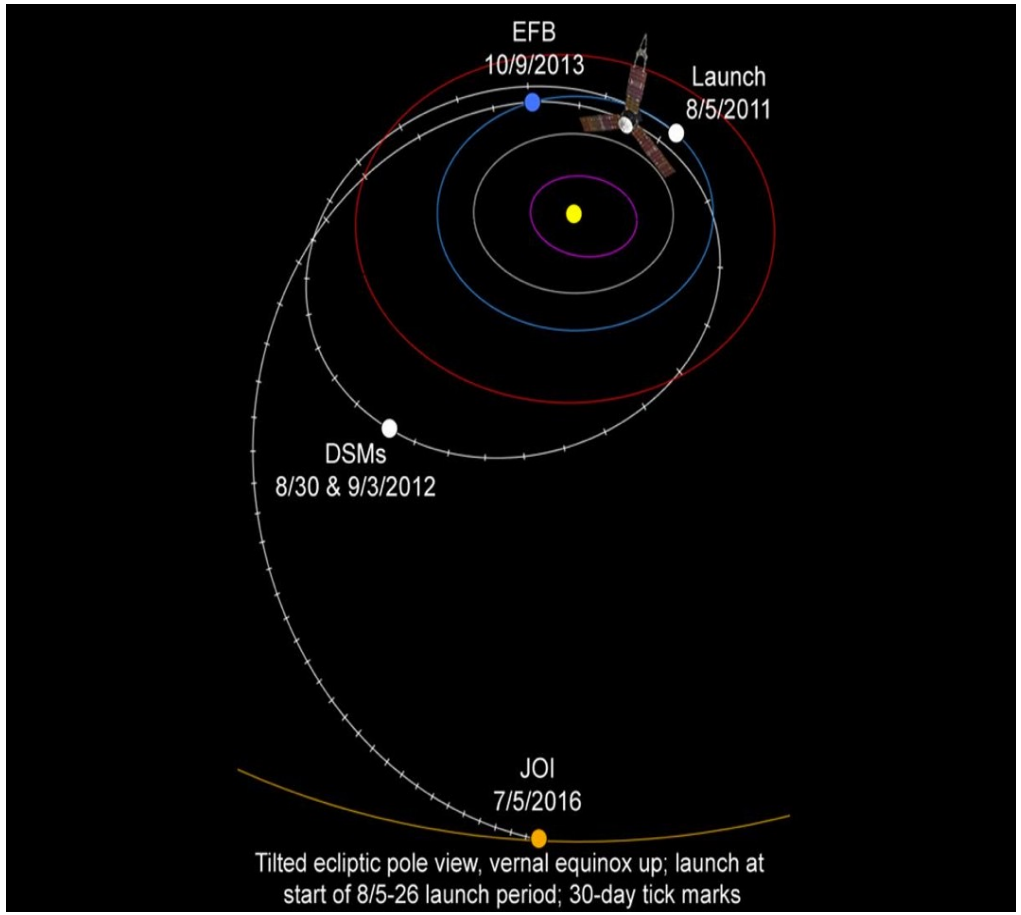
1. **LAUNCH**
   Astronauts lift off from pad 39B at Kennedy Space Center.
2. **JETTISON ROCKET BOOSTERS, FAIRINGS, AND LAUNCH ABORT SYSTEM**
3. **CORE STAGE MAIN ENGINE CUT OFF**
   With separation.
4. **PERIGEE RAISE MANEUVER**
5. **APOGEE RAISE BURN TO HIGH EARTH ORBIT**
   Begin 24 hour checkout of spacecraft.
6. **PROX OPS DEMONSTRATION**
   Orion proximity operations demonstration and manual handling qualities assessment for up to 2 hours.
7. **INTERIM CRYOGENIC PROPULSION STAGE (ICPS) DISPOSAL BURN**
8. **HIGH EARTH ORBIT CHECKOUT**
   Life support, exercise, and habitation equipment evaluations.
9. **TRANS-LUNAR INJECTION (TLI) BY ORION'S MAIN ENGINE**
   Lunar free return trajectory initiated with European service module.
10. **OUTBOUND TRANSIT TO MOON**
    4 days outbound transit along free return trajectory.
11. **LUNAR FLYBY**
    4,000 nmi (mean) lunar farside altitude.
12. **TRANS-EARTH RETURN**
    Return Trajectory Correction (RTC) burns as necessary to aim for Earth's atmosphere; travel time approximately 4 days.
13. **CREW MODULE SEPARATION FROM SERVICE MODULE**
14. **ENTRY INTERFACE (EI)**
    Enter Earth's atmosphere.
15. **SPLASHDOWN**
    Ship recovers astronauts and capsule.



PROXIMITY OPERATIONS DEMONSTRATION SEQUENCE

Motivation: Juno





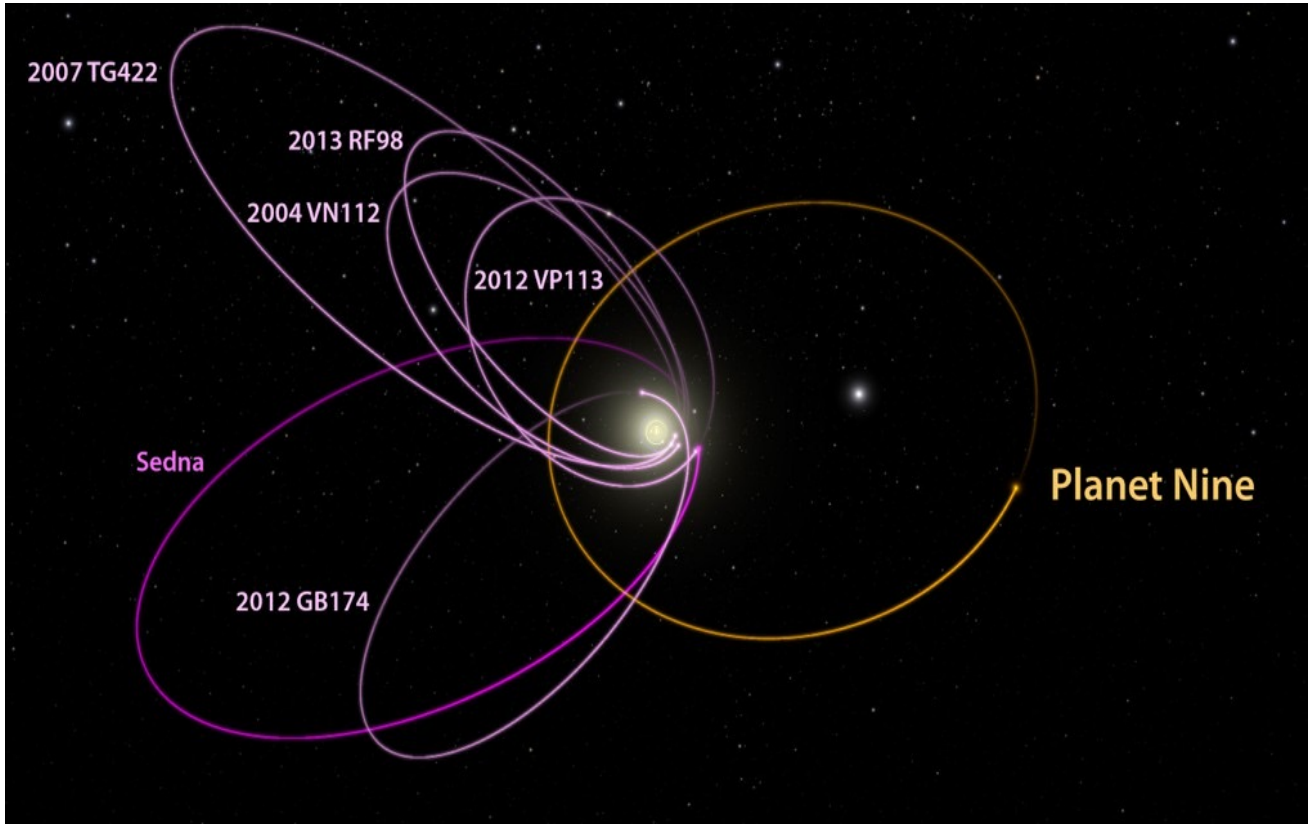
More information: https://science.nasa.gov/mission/juno/

Motivation: Planet 9

“Evidence For A Distant Giant Planet In The Solar System.”

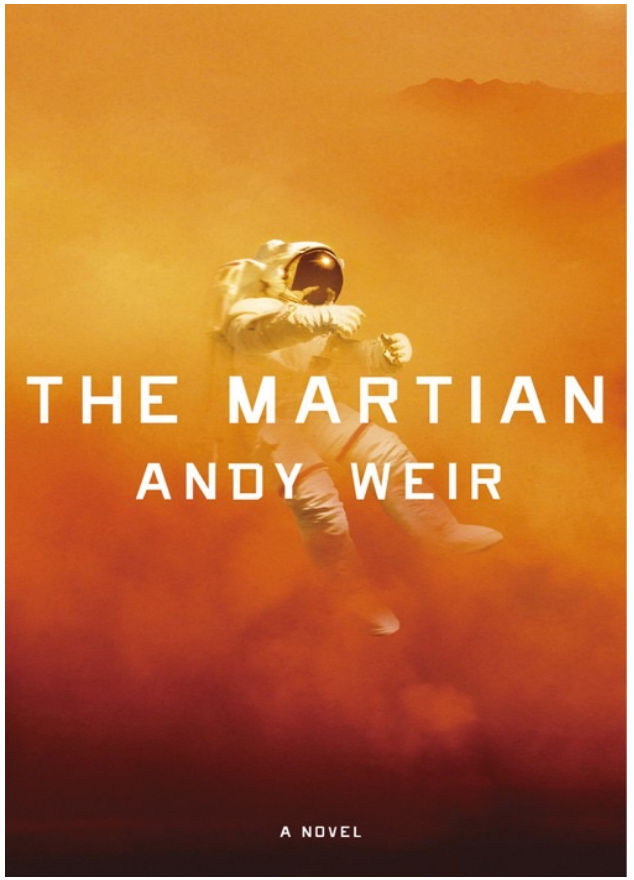
Konstantin Batygin and Mike Brown, discovered the planet's existence through mathematical modeling and [N-body] computer simulations.

https://www.caltech.edu/about/news/caltech-researchers-find-evidence-real-ninth-planet-49523



Motivation: Matt Damon

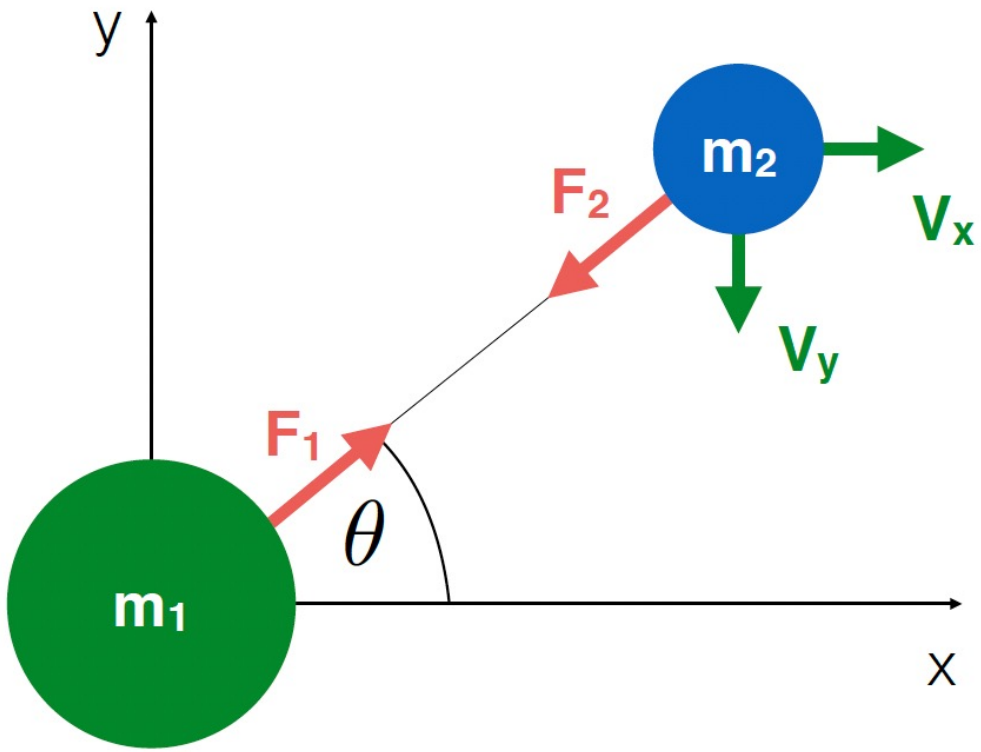




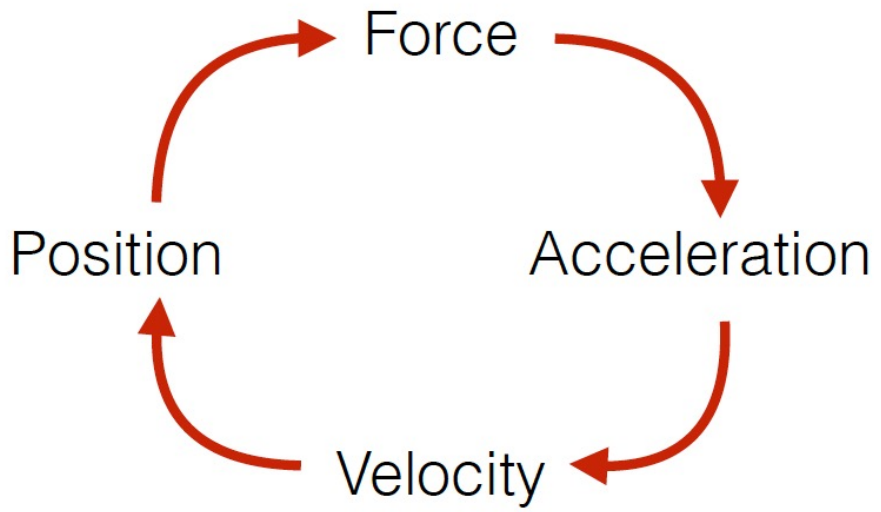
Final Project

Your objective for the final project is to develop a tool that will allow you to **simulate your own three-dimensional, gravity-driven solar system**. Although there is only one simple equation governing all these interactions, successfully coding this simulation will require some careful thinking and consideration of everything we've done this semester.

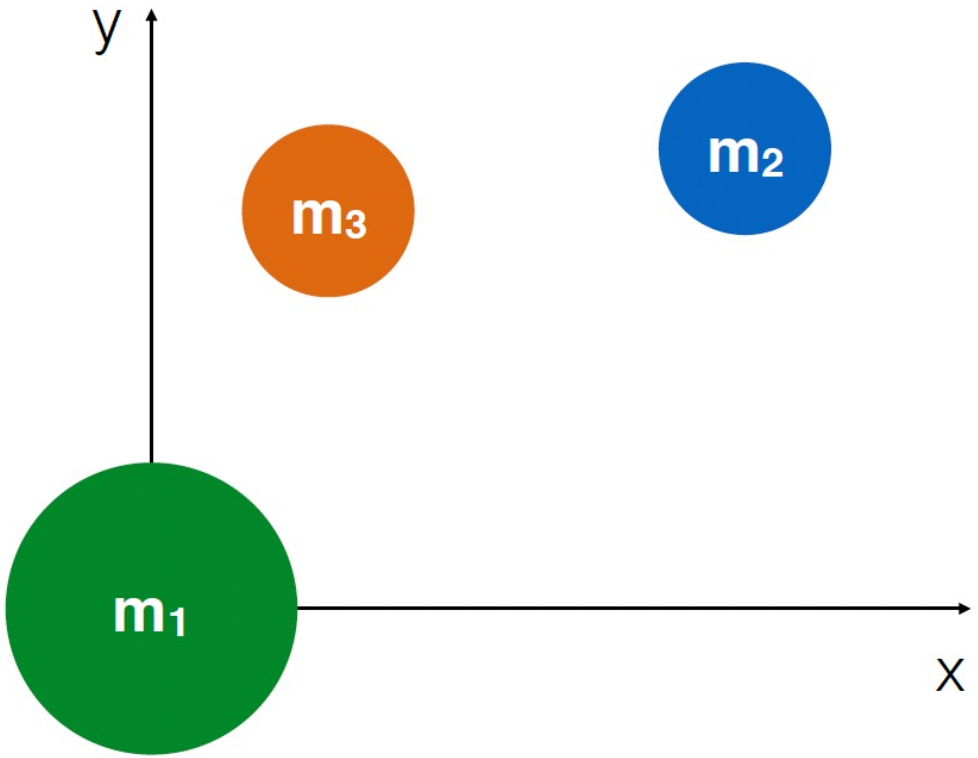
Two-Body Problem



$$F_1 = F_2 = G\frac{m_1 \cdot m_2}{r^2}$$

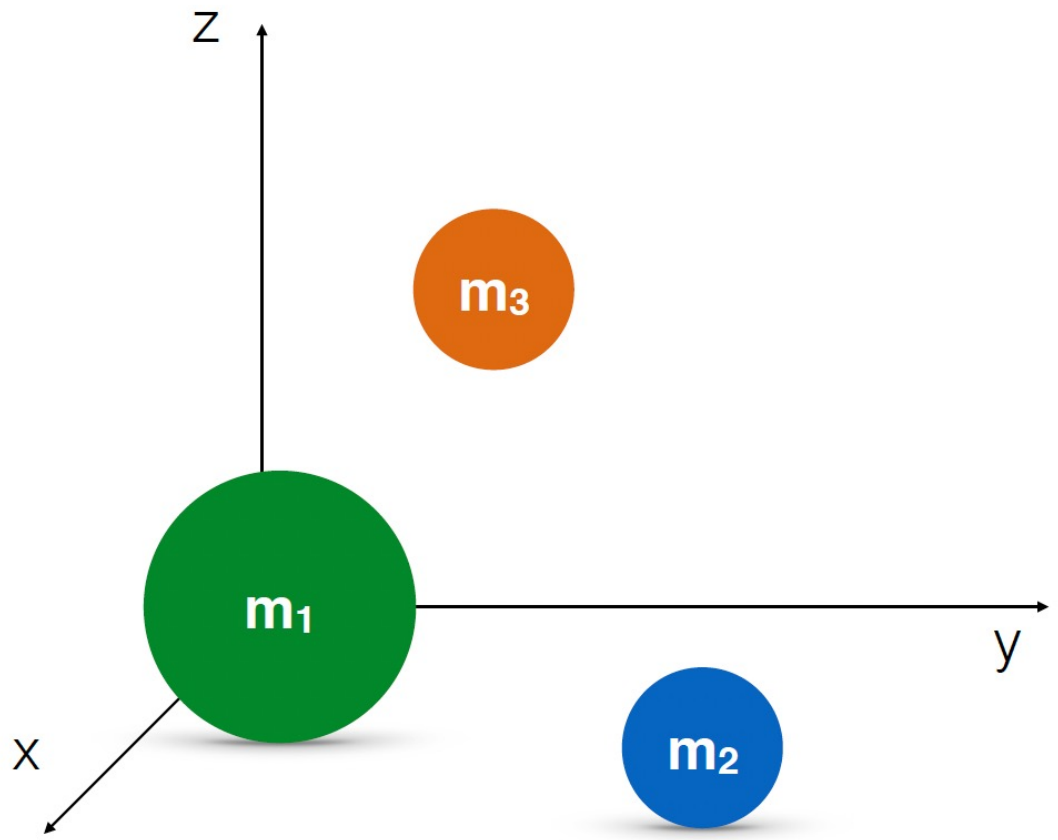


Three-Body Problem



How does our solution change if we add another object?

3D Three-Body Problem

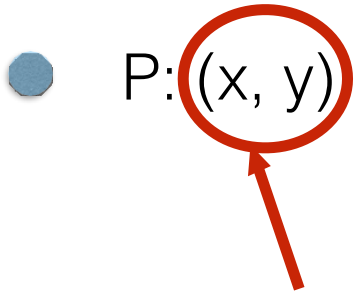


How does our solution change if we add another dimension?

First, let's think about how we'll store the data for this problem.

Structures

Example: A point in the plane.



Every point has an x coordinate and a y coordinate

Structures

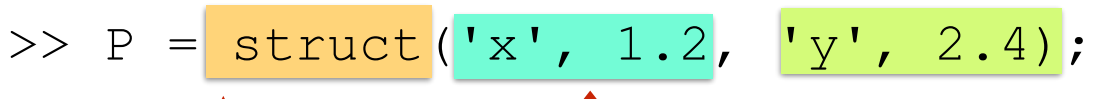
Example: A point in the plane.

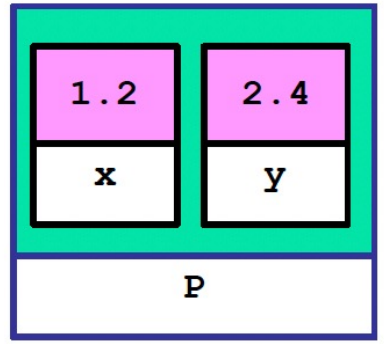
P: (1.2, 2.4)



It would be easier to refer to the point with a single variable that "contains" the coordinates.

We can achieve this with a **structure**





This creates the structure

Creates a field called 'x' and assigns the value 1.2 to it

Creates another field called 'y' and assigns the value 2.4 to it

```
>> P = struct('x', 1.2, 'y', 2.4);
```

We can access the data easily:

```
>>  P
P   =

        x :   1.2000
        y :   2.4000

>> P.x

ans =

1.2000

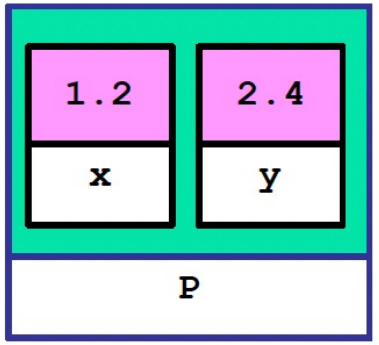
>> P.y

ans =

2.4000
```

Fields of structure are accessed using **.**

P is a *structure*.
It has two *fields*. Their names are **x** and **y**.

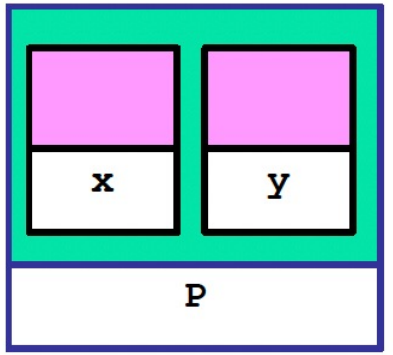


In fact, we could have used a slightly easier notation to create the fields

```
>> P.x = 1.2; P.y = 2.4;
```

Or, if we just want to create the fields but assign their values later:

```
>> P.x = []; P.y = [];
```



We can construct arrays of structures

- Each element of the array is a structure
- All of the structures must be 'similar' (i.e. have the same fields)

```
>> Pts(1:5) = struct('x',[],'y',[]);
>> Pts(3)
ans =
       x: []
       y: []
>> Pts(3).x = 4; Pts(3).y = -2;
>> Pts(3)
ans =
       x: 4
       y: -2
```



Create 5 element array of structures, each with fields x and y

When working with structures…

It is good practice to write a function that makes the structure.
This ensures uniformity.

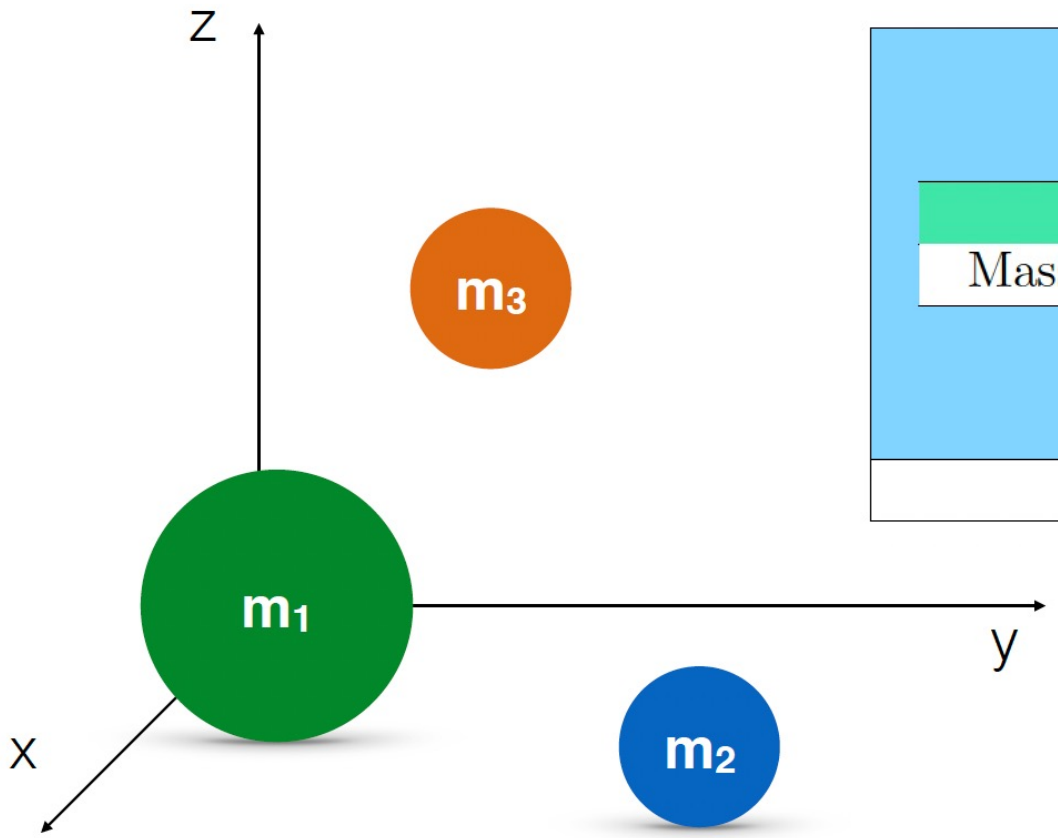
```
function [ P ] = MakePoint( x, y )
%MAKEPOINT Create a point in the plane
%   P = MAKEPOINT(x,y) creates a point structure from the specified
%   coordinates x and y. These are stored in fields P.x and P.y,
%   respectively.

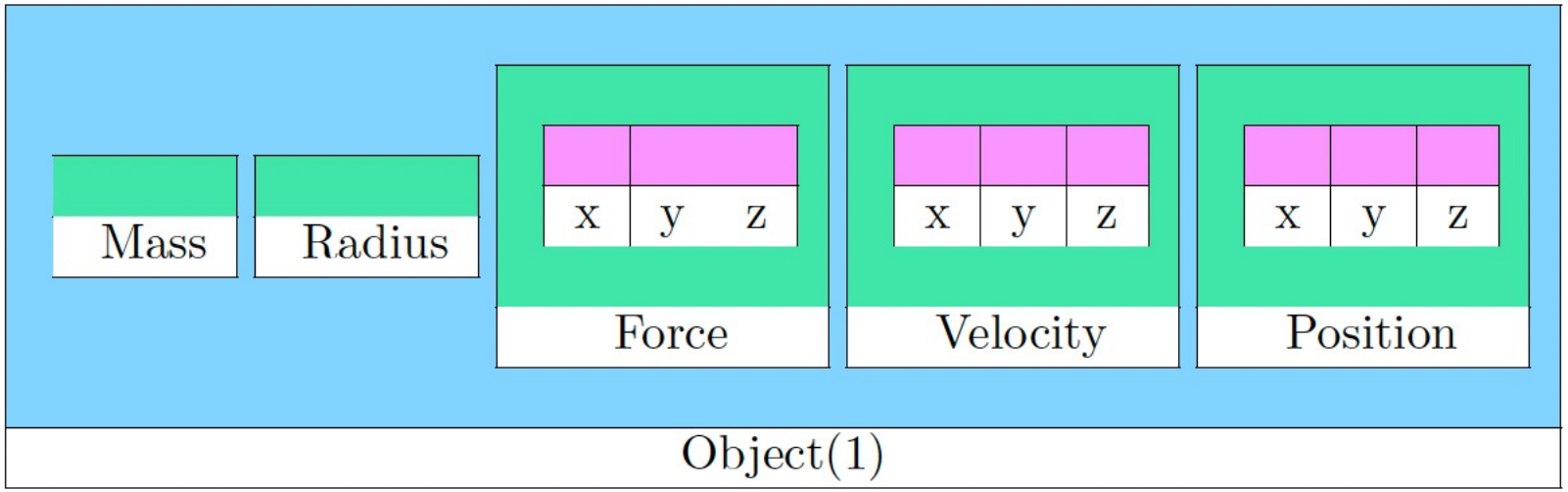
P = struct('x',x,'y',y);

end
```

Structures in Final Project

How can we apply these concepts to the final project?



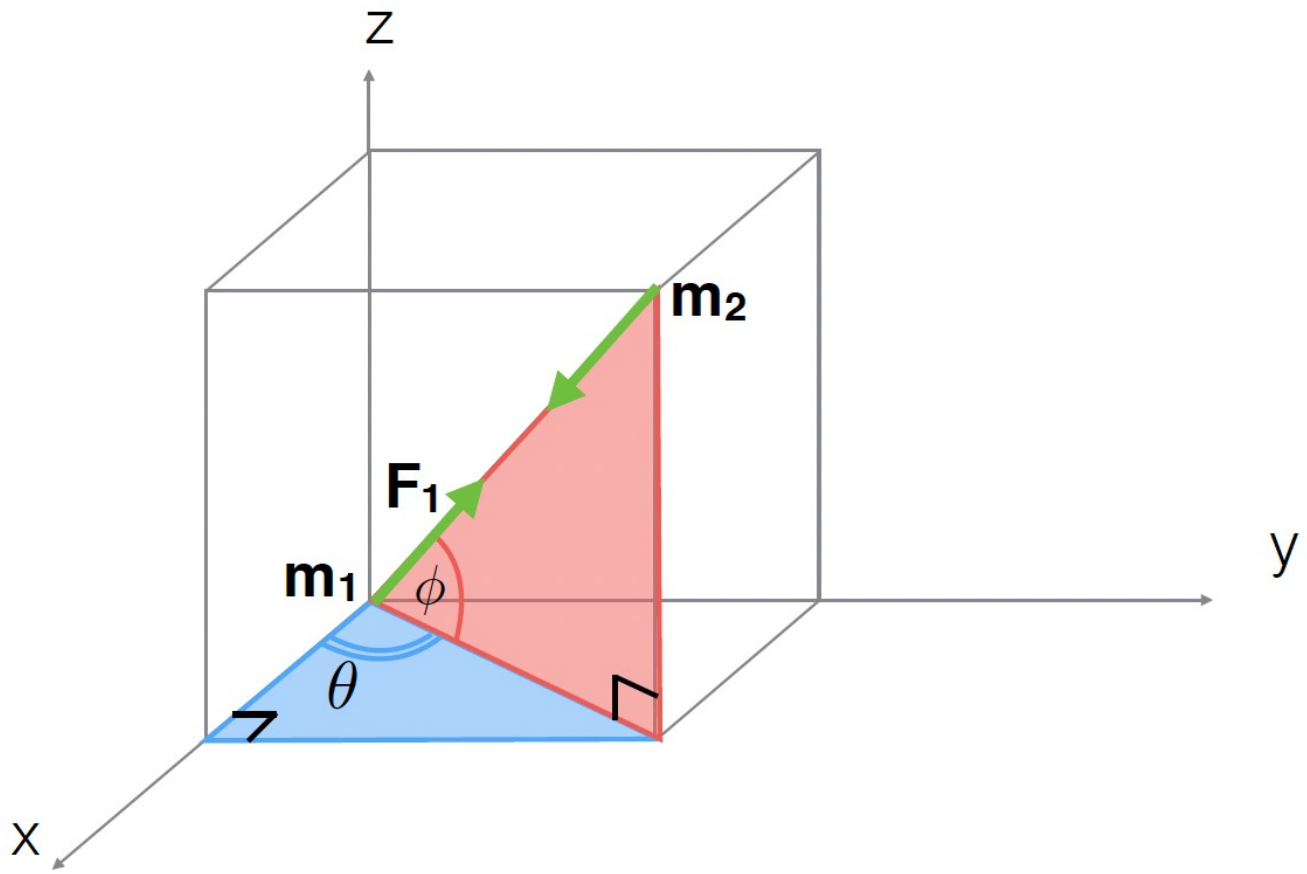



```
>> Object(1).Force.x

>> Object(2).Radius

>> Object(N).Position.z
```

Forces in 3D

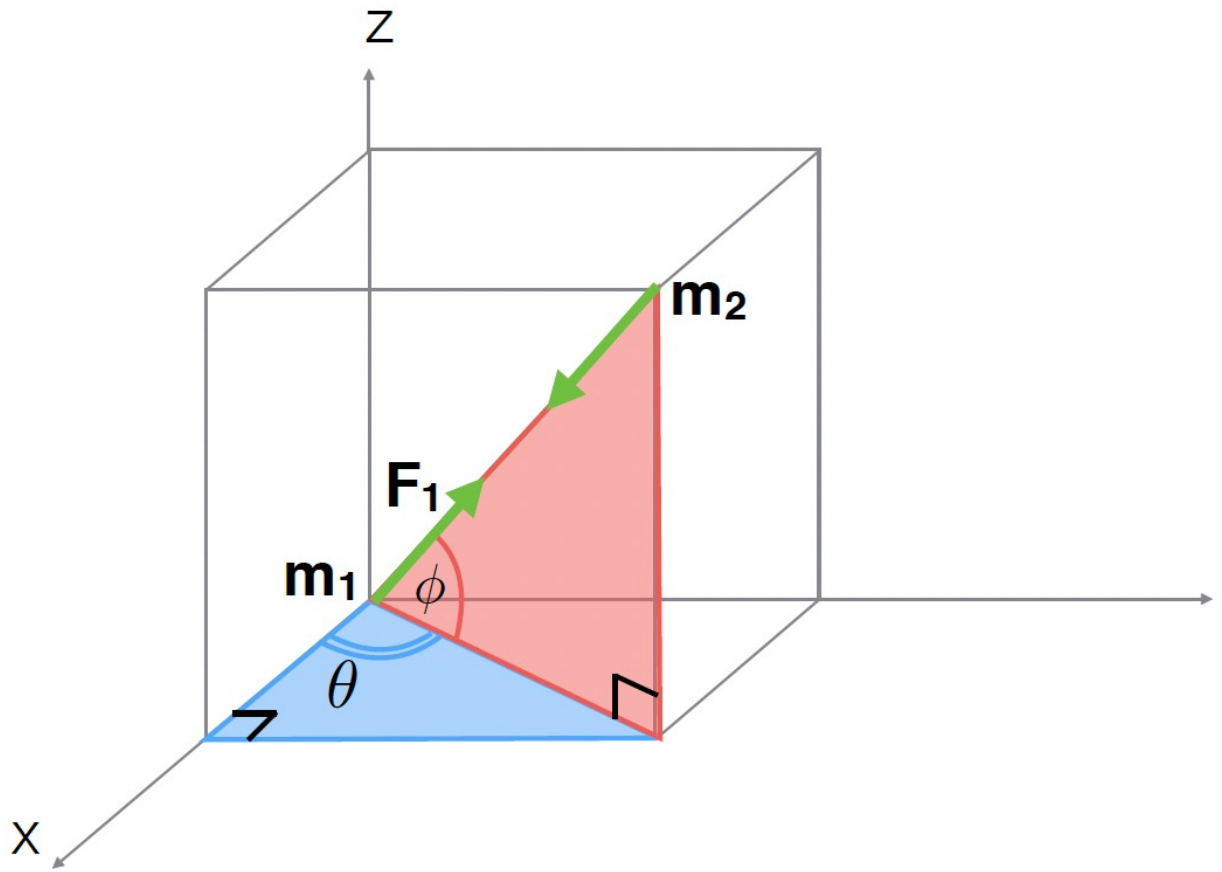


$$\mathbf{F}_1 = G\frac{m_1 \cdot m_2}{\mathbf{r}^2}$$

Remember, we need to take into account the interaction with all ***N*** bodies.

$$\mathbf{F}_1 = G\sum_{j=2}^{N}\frac{m_1 \cdot m_j}{\mathbf{r}^2}$$

Forces in 3D

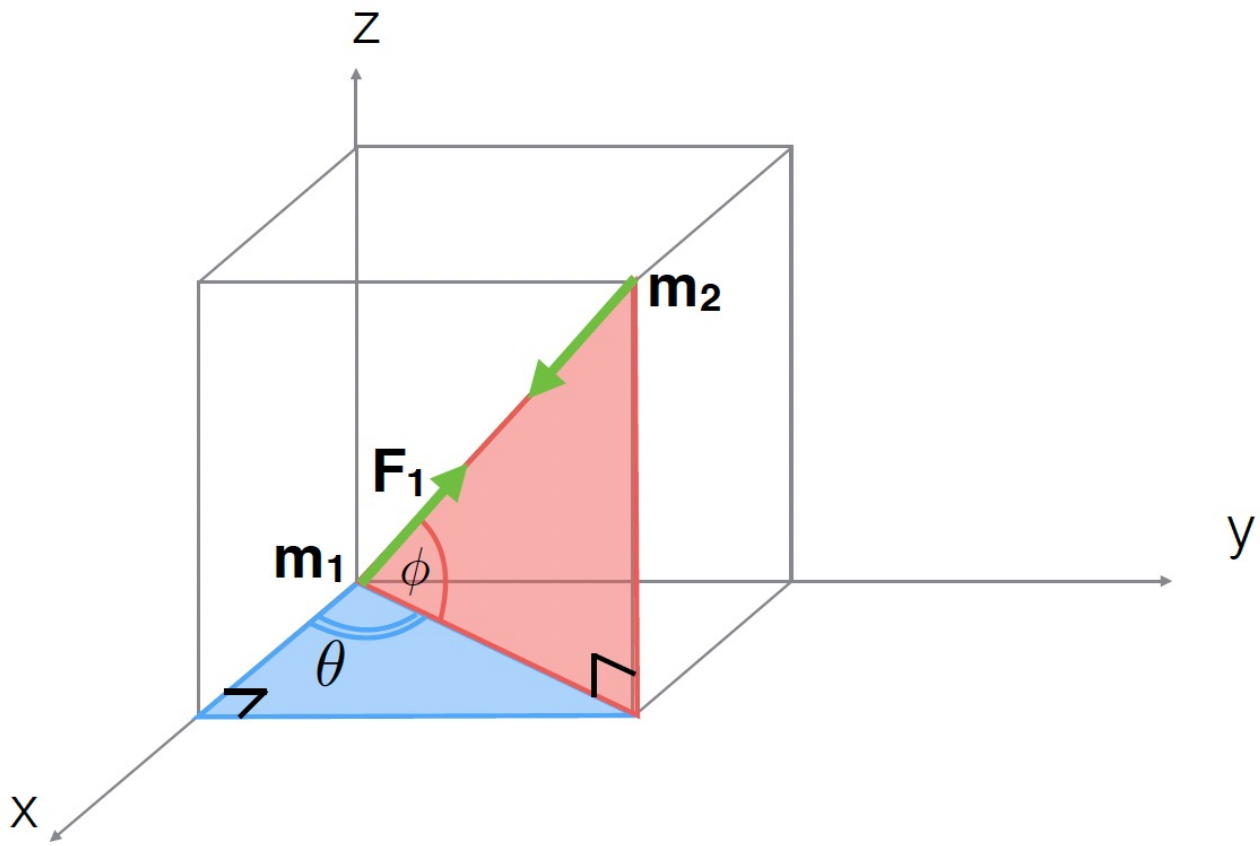


Or more generally,

$$\mathbf{F}_i = G \sum_{j=1}^{N} \frac{m_i \cdot m_j}{\mathbf{r}^2}$$

where $j \neq i$

Force Components in 3D



We can either compute an **elevation** and **rotation** angle, or directly use the information in our vectors (easier).

$$\mathbf{F}_i = G \sum_{j=1}^{N} \frac{m_i \cdot m_j}{\mathbf{r}^2}$$

where $j \neq i$

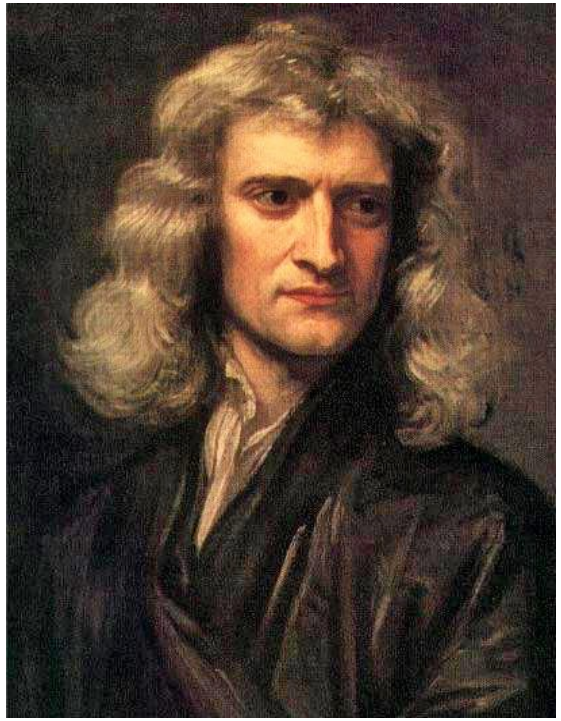
$$F_i^x = G \sum_{j=1}^{N} \frac{m_i \cdot m_j}{\mathbf{r}^3} r_x$$

$$F_i^y = G \sum_{j=1}^{N} \frac{m_i \cdot m_j}{\mathbf{r}^3} r_y$$

$$F_i^z = G \sum_{j=1}^{N} \frac{m_i \cdot m_j}{\mathbf{r}^3} r_z$$

Kinematic Updates

Now that we've determined the forces on objects **1:N**, we can update their motion.



Newton's Law II

The change of motion is proportional to the motive force impressed; and is made in the direction of the right line in which that force is impressed.

$$\boldsymbol{F} = m \cdot \boldsymbol{a}$$

Kinematic Updates

Now that we've determined the forces on objects **1:N**, we can update their motion.

Let's use `object(1)` as an example:

$$\mathbf{F_1} = m_1 \cdot \mathbf{a_1}$$

$$\Rightarrow \mathbf{a_1} = \frac{\mathbf{F_1}}{m_1}$$

Remember, we're dealing with multiple components for **F** and **a**, for example:

$$\mathbf{a_1} \equiv \begin{bmatrix} a_1^x \\ a_1^y \\ a_1^z \end{bmatrix}$$

Kinematic Updates

Our equations of motion can be written as

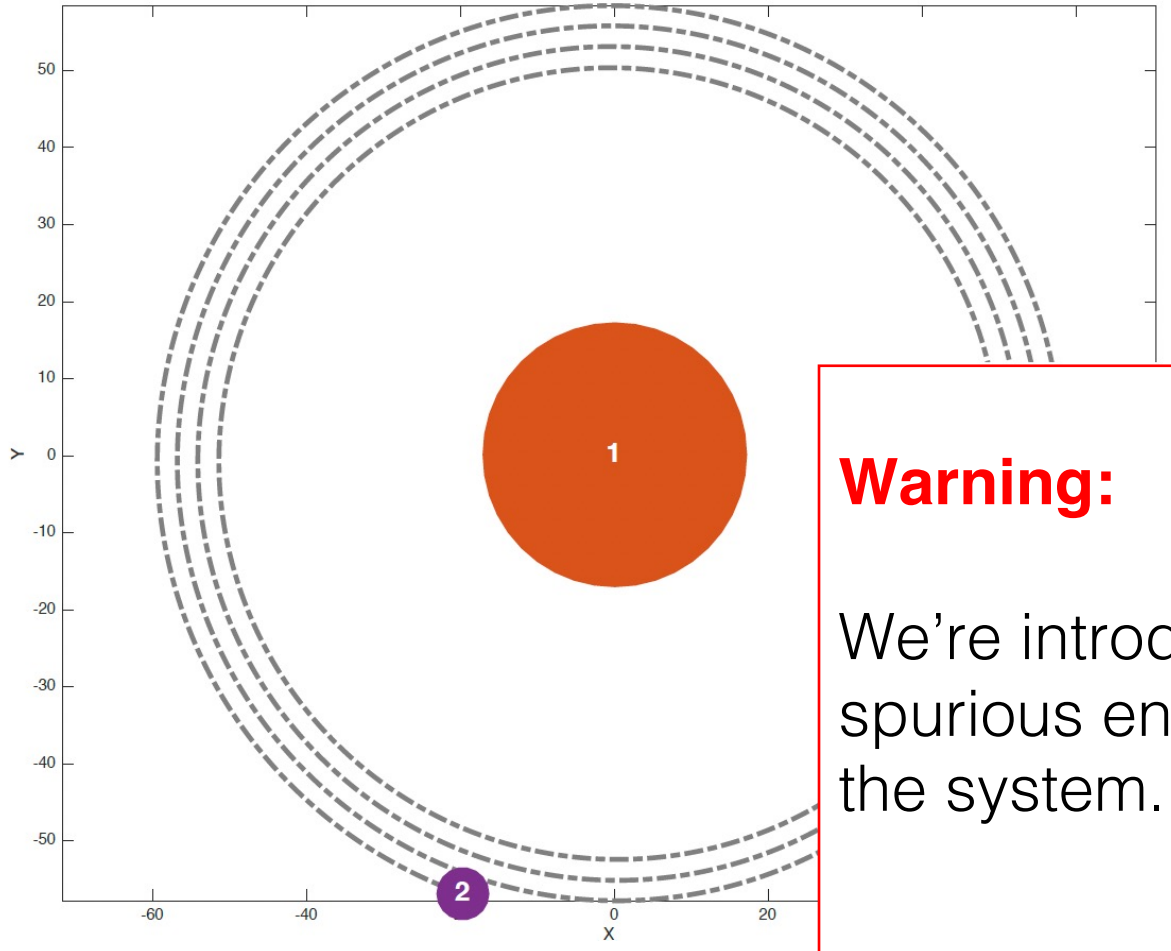
$$\frac{d\mathbf{v}}{dt} = \mathbf{a} \qquad \frac{\mathbf{v}(k+1) - \mathbf{v}(k)}{\Delta t} = \mathbf{a}(k)$$

Explicit Euler!

$$\frac{d\mathbf{x}}{dt} = \mathbf{v} \qquad \frac{\mathbf{x}(k+1) - \mathbf{x}(k)}{\Delta t} = \mathbf{v}(k)$$

We can re-arrange to solve for the **velocity** and **position** at the next time step, **k+1**.

Explicit Euler



Warning:

We're introducing spurious energy into the system.

Kinematic Updates

Explicit Euler didn't work, so let's try something else

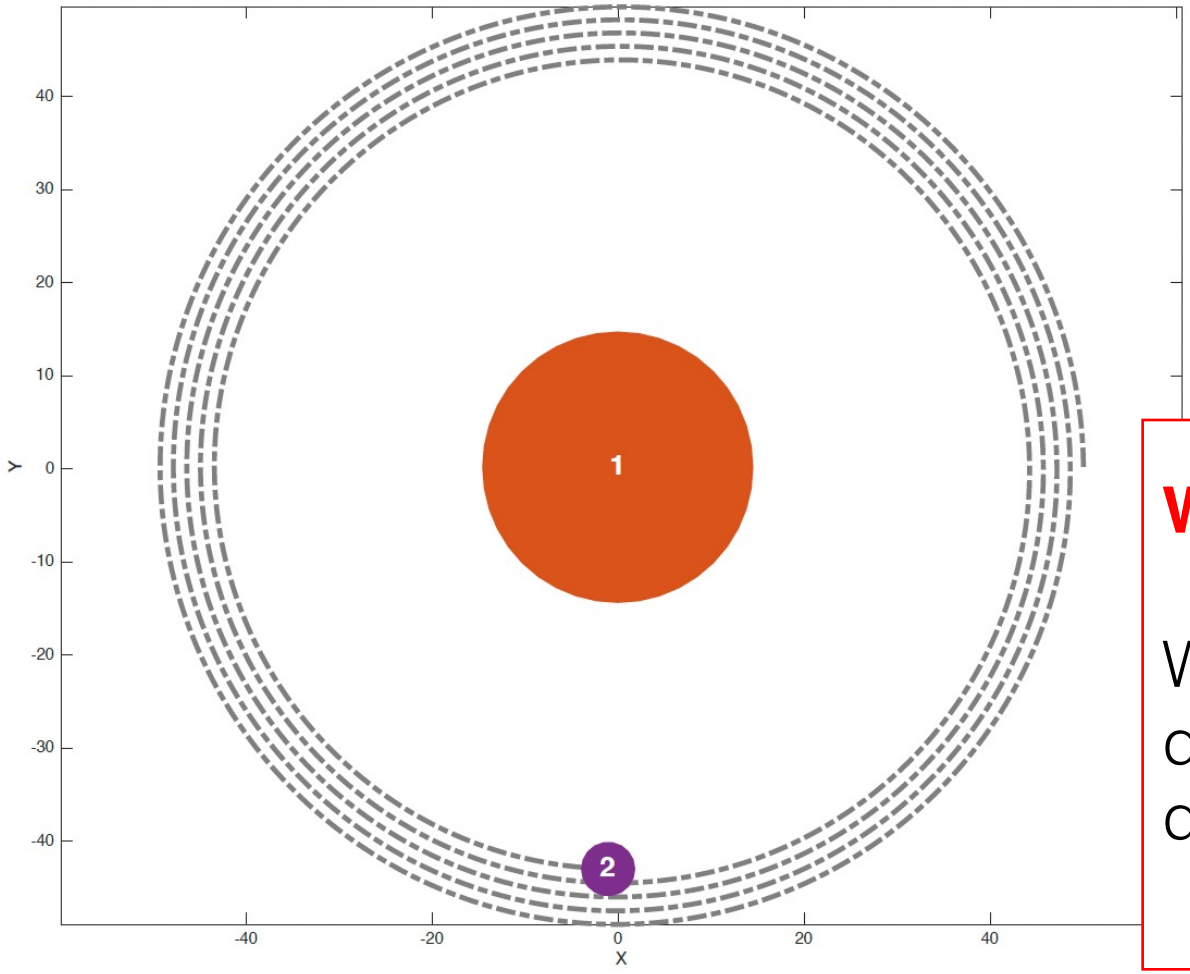
$$\frac{d\mathbf{v}}{dt} = \mathbf{a} \qquad \frac{\mathbf{v}(k+1) - \mathbf{v}(k)}{\Delta t} = \mathbf{a}(k+1)$$

Implicit Euler!

$$\frac{d\mathbf{x}}{dt} = \mathbf{v} \qquad \frac{\mathbf{x}(k+1) - \mathbf{x}(k)}{\Delta t} = \mathbf{v}(k+1)$$

As before, we can re-arrange to solve for the **velocity** and **position** at the next time step, **k+1**.

Implicit Euler



Warning :

We are losing energy due to excessive damping.

Kinematic Updates

Implicit Euler didn't work either.

We need a compromise between **Explicit** and **Implicit**.

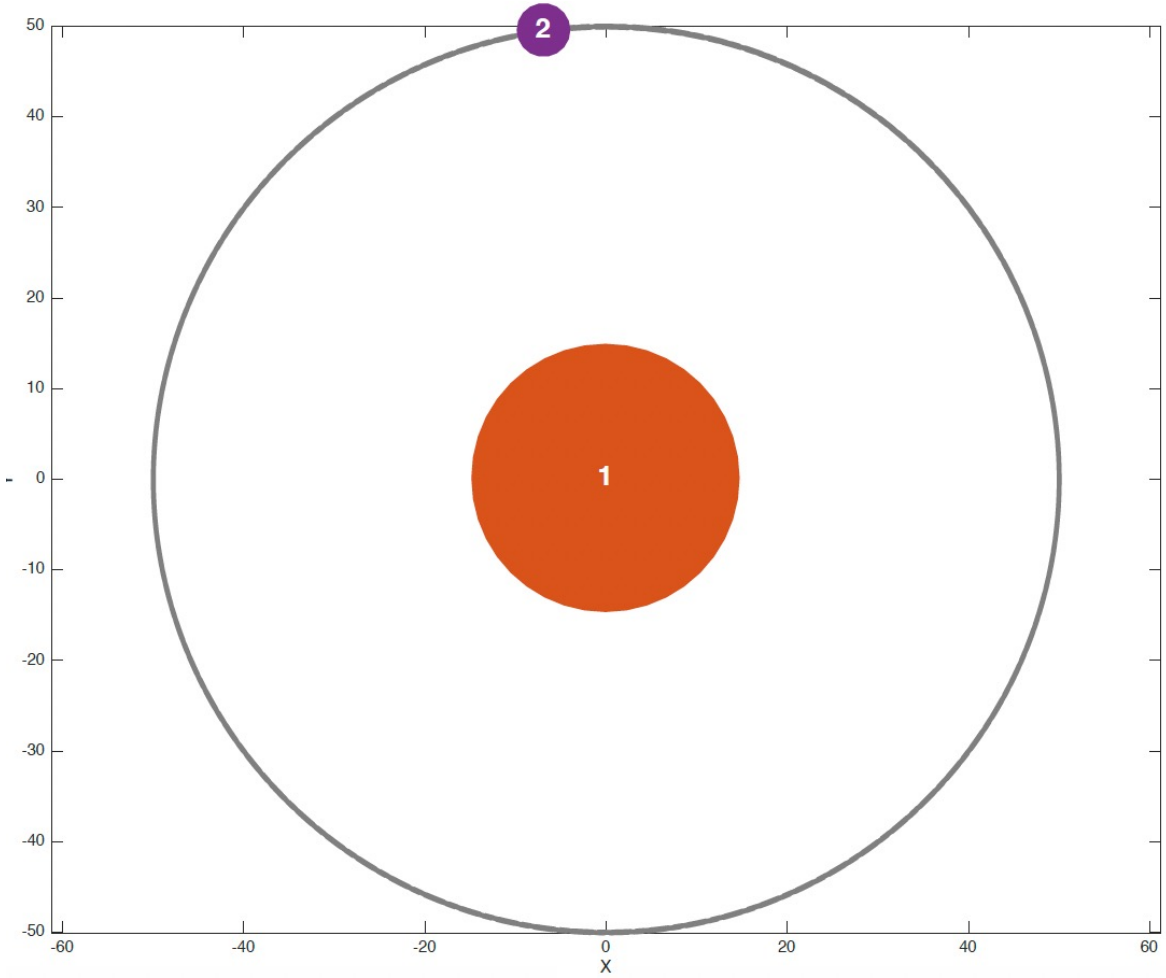
$$\frac{d\mathbf{v}}{dt} = \mathbf{a} \qquad \frac{\mathbf{v}(k+1) - \mathbf{v}(k)}{\Delta t} = \mathbf{a}(k)$$

Semi-Implicit Euler!

$$\frac{d\mathbf{x}}{dt} = \mathbf{v} \qquad \frac{\mathbf{x}(k+1) - \mathbf{x}(k)}{\Delta t} = \mathbf{v}(k+1)$$

The position is calculated using the "new" velocity at timestep **k+1**.

Semi-Implicit Euler



Now it's good!

Conserves energy on average!😀

Putting it all together

1. **Set up the structured variable array**

2. **Calculate the total force on each object**

3. **Calculate the acceleration; update velocity and position**

4. **Save the results to a movie file (you will learn it on 04/28)**

For this and next week, no worries about generating any movie file – you can just view the animation using a combination of `plot3()` and `drawnow`.